

MANUALE TECNICO



Monticone Matteo, De Mutiis Damiano, Porto Bonacci Matteo,
Tamburo Mattia

24 maggio 2026

Versione 2.0

Indice

1. Installazione.....	3
1.1 Requisiti di sistema.....	3
1.2 Setup ambiente.....	3
1.3 Installazione programma.....	3
2. Scelte Progettuali.....	4
2.1 Documento di analisi dei requisiti.....	4
2.2 Schema ER iniziale.....	5
2.3 Schema ER ristrutturato.....	8
2.4 Schema relazionale.....	9
2.5 Class diagram UML.....	10
2.6 Use case diagram UML.....	13
2.7 Package diagram UML.....	14
2.8 State diagram UML.....	15
3. Scelte Architettureali.....	17
3.1 Interfaccia grafica.....	17
3.2 Librerie.....	19
3.3 Strutture dati utilizzate.....	21
3.4 Oggetti di supporto al database.....	22
3.5 Gestione degli errori e logging.....	23
3.6 Cifratura dei dati.....	23
4. Scelte algoritmiche.....	25
4.1 Login.....	25
4.2 Home.....	27
4.3 Restaurant GUI.....	28
4.4 InputPattern.....	29

4.5 Controller.....	33
4.6 Aggiunta di recensioni e risposte.....	34
4.7 Server.....	35
4.8 ServerHandler.....	37
5. Formato dei file e gestione.....	38
5.1 Formato file.....	38
5.2 AppPaths.....	38
5.3 Dataset di test.....	38
5.4 Gestione dei file.....	39
6. Limitazioni della soluzione sviluppata.....	41

Installazione

1.1 Requisiti di sistema

Qui elencati i requisiti minimi necessari per il corretto funzionamento del programma sul proprio dispositivo:

Sistema operativo: Windows 10, macOS 11, Ubuntu 20.04 LTS

CPU: Processore Dual-Core a 64-bit

Spazio di memoria richiesto: 1 GB

RAM: 4 GB

Risoluzione minima: 1024x768

1.2 Setup ambiente

Per il funzionamento del programma è necessario installare nel proprio dispositivo la versione Java 19 (o successive).

Tale versione è scaricabile sul sito di Oracle nella sezione Downloads (raggiungibile tramite il link: <https://www.oracle.com/it/java/technologies/downloads/>).

Per quanto riguarda l'utilizzo del database invece è consigliata l'installazione di PostgreSQL (raggiungibile tramite il link: <https://www.postgresql.org/>).

1.3 Installazione programma

Una volta scaricato il pacchetto di installazione, aprire il file eseguibile (jar) per dare il via al funzionamento del programma lato client.

Lato server invece è necessario eseguire il file serverTK.jar da linea di comando tramite l'istruzione: "java -jar serverTK.jar" (questo sempre dopo essersi correttamente spostati nella directory contenente il file jar).

Scelte progettuali

2.1 Documento di analisi dei requisiti

Prima della progettazione del Database vero e proprio, ne è stata definita una struttura che possa permettere di memorizzare tutte le informazioni necessarie al corretto funzionamento dell'applicazione.

Le principali informazioni che devono essere gestite riguardano gli utenti registrati, le recensioni pubblicate da essi ed i ristoranti. Deve perciò essere resa più semplice possibile la ricerca di uno o più ristoranti da parte di un utente in base a determinate caratteristiche.

Come da specifiche, è necessario realizzare una tabella che memorizzi i dati riguardanti i ristoranti. Per ognuno di questi è fondamentale memorizzarne:

- Nome
- Nazione
- Città
- Indirizzo completo
- Latitudine
- Longitudine
- Fascia di prezzo (prezzo medio espresso nella valuta indicata nel campo currency)
- Disponibilità del servizio di delivery (sì/no)
- Disponibilità del servizio di prenotazione online (sì/no)
- Tipologia di cucina offerta

Si tratta di dati vitali per permettere un preciso e soprattutto utile filtraggio dei ristoranti in fase di ricerca da parte degli utenti.

Altra tabella per cui sono state fornite precise direttive è la tabella riguardante gli utenti, necessaria per gestire gli account di tutti gli utilizzatori della piattaforma. I dati da memorizzare in questo caso sono:

- Nome
- Cognome
- Username/E-mail
- Password (che deve essere ovviamente cifrata)
- Data di nascita (facoltativo)
- Luogo del domicilio
- Ruolo (cliente/ristoratore)

E' importante specificare che il ruolo consente di distinguere il cliente (che può consultare i ristoranti e scrivere delle recensioni) e il ristorante (che può creare un ristorante e o modificare/aggiornare le informazioni relative al suo ristorante).

L'ultima tabella espressamente citata all'interno delle specifiche del progetto è quella riguardante le recensioni. Ogni recensione deve dunque essere associata ad un utente registrato (autore) e ad un ristorante presente nel database. La tabella recensioni perciò potrebbe quasi essere vista come un collegamento tra la tabella degli utenti e quella dei ristoranti.

In conclusione, il database realizzato deve permettere una corretta organizzazione dei dati (specialmente riguardo gli utenti in quanto ne esistono di due distinte tipologie, con due distinte possibilità di interazione con l'applicazione), una buona facilità di consultazione (in quanto la ricerca dei ristoranti per caratteristiche è una delle colonne portanti dell'applicazione) e sicurezza per le informazioni sensibili (come per esempio la password cifrata).

2.2 Schema ER iniziale

Lo schema ER rappresentante la struttura logica del database è stato progettato per gestire informazioni relative agli utenti, ai ristoranti e alle recensioni come base primaria di tutto il sistema.

L'entità centrale del diagramma è Restaurant (che conterrà ovviamente tutte le informazioni inerenti ai ristoranti). I suoi attributi saranno un identificativo (posto come chiave primaria) fondamentale per il riconoscimento del ristorante nel sistema, attributi descrittivi (come il nome, il sito web, la fascia di prezzo e la valutazione) ed infine attributi legati alla collocazione geografica.

L'entità User invece rappresenta e raccoglie tutti gli utenti registrati, salvando le informazioni fornite dall'utente in fase di accesso e assegnando anche in questo caso uno specifico identificativo (molto utile per il riconoscimento veloce dell'utente nel sistema). Dallo schema si può notare una suddivisione di User in due differenti sottocategorie quali:

- Customer → cliente che utilizza l'applicazione per ricercare ristoranti e pubblicarne in seguito recensioni;
- Restaurateur → ristorante a capo di uno o più ristoranti.

Per quanto riguarda le recensioni invece, nello schema è stata introdotta l'entità Review che ha lo scopo di memorizzare le recensioni scritte dagli utenti. Ogni recensione, come per le precedenti entità, possiede un personale identificativo ed in questo caso anche una valutazione numerica ed il suo contenuto testuale. Inoltre è

presente anche un'entità Response che ha il semplice scopo di contenere le risposte date dai ristoratori alle recensioni degli utenti.

All'interno del diagramma vengono introdotte anche due entità rispettivamente chiamate Cuisine e Service che sono utili per la descrizione delle tipologie di cucina offerte dai ristoranti e per elencare i servizi disponibili (un esempio sarebbe il delivery).

Per quanto riguarda le relazioni invece abbiamo:

- Possiede → collega il ristoratore con i ristoranti e indica che un gestore potrebbe possedere più di un ristorante, mentre invece un ristorante può avere un solo proprietario.
- Scrive → collega il cliente con le recensioni indicando appunto la possibilità di scrivere valutazioni personali relative ad un ristorante.
- Ha → collega le recensioni al ristorante a cui si riferiscono permettendo in questo modo di avere più recensioni per un singolo ristorante.
- Tipologie offerte → collega il ristorante alla tipologia di cucina offerta.
- Fornisce → collega il ristorante ai servizi offerti.
- Aggiunge ai preferiti → collega il cliente al ristorante facilitando così il salvataggio dei ristoranti preferiti all'interno della propria lista preferiti.
- Risponde → collega la recensione alla risposta data dal ristoratore (se scritta).
- Scritta da → collega il ristoratore che scrive la risposta ad una recensione al contenuto effettivo della risposta.

Il diagramma ER perciò è strutturato nel modo visibile nella pagina seguente.

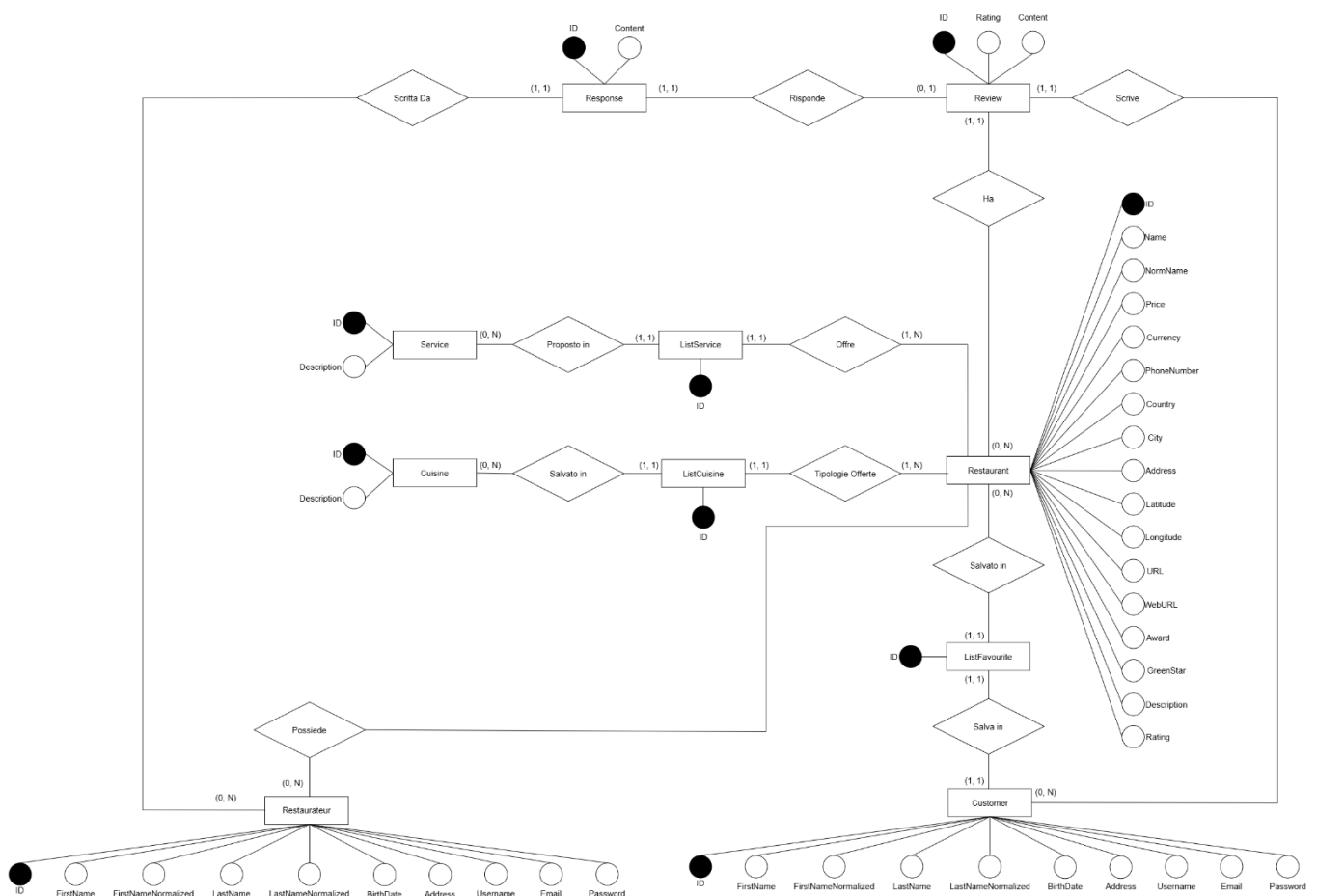
2.3 Schema ER ristrutturato

Il diagramma precedentemente illustrato non è ancora al suo massimo livello di ottimizzazione e perciò necessita di una fase di ristrutturazione.

Per prima cosa è stata gestita la gerarchia di User, dove si è deciso di mantenere una separazione netta tra le entità Customer e Restaurateur per via delle differenze sostanziali che i due tipi di utenti hanno all'interno del sistema. Sono stati comunque mantenuti gli attributi iniziali di User e semplicemente assegnati ad entrambe le entità.

Successivamente sono state ristrutturate le relazioni multi-a-molti:

- Le entità Cuisine e Service erano direttamente collegate ai ristoranti, durante la ristrutturazione sono state introdotte le entità intermedie ListCuisine e ListService utili ad evitare ridondanze nei dati (viene permesso infatti di associare un ristorante direttamente a più tipologie di cucina e servizi).
- Il salvataggio di un ristorante nella lista preferiti da parte di un cliente è stato riorganizzato introducendo l'entità ListFavourite. In questo modo viene garantita una maggiore semplicità nel salvataggio dei ristoranti preferiti dai clienti all'interno di una opportuna lista.



2.4 Schema relazionale

In conclusione alla fase di progettazione del database, una volta prodotto un diagramma ER correttamente ristrutturato è stato possibile procedere nella stesura dello schema relazionale del database.

La sua struttura è la seguente:

Customer (ID (PRIMARY KEY), firstName, firstNameNormalized, lastName, lastNameNormalized, birthDate, address, username, email, password)

Restaurateur (ID (PRIMARY KEY), firstName, firstNameNormalized, lastName, lastNameNormalized, birthDate, address, username, email, password)

Restaurant (ID (PRIMARY KEY), ownerID (FOREIGN KEY a Restaurateur(ID)), name, normName, price, currency, phoneNo, country, city, address, latitude, longitude, url, webUrl, award, greenStar, description, rating)

Review (ID (PRIMARY KEY), restaurantID (FOREIGN KEY a Restaurant(ID)), IDCustomer (FOREIGN KEY a Customer (ID)), rating, content)

Cuisine (ID (PRIMARY KEY), description)

Service (ID (PRIMARY KEY), description)

ListFavourite ((restaurantID (FOREIGN KEY a Restaurant(ID)), customerID (FOREIGN KEY a Customer(ID))) (PRIMARY KEY))

ListCuisine ((restaurantID (FOREIGN KEY a Restaurant(ID)), cuisineID (FOREIGN KEY a Cuisine (ID))) (PRIMARY KEY))

ListService ((restaurantID (FOREIGN KEY a Restaurant(ID)), serviceID (FOREIGN KEY a Service (ID))) (PRIMARY KEY))

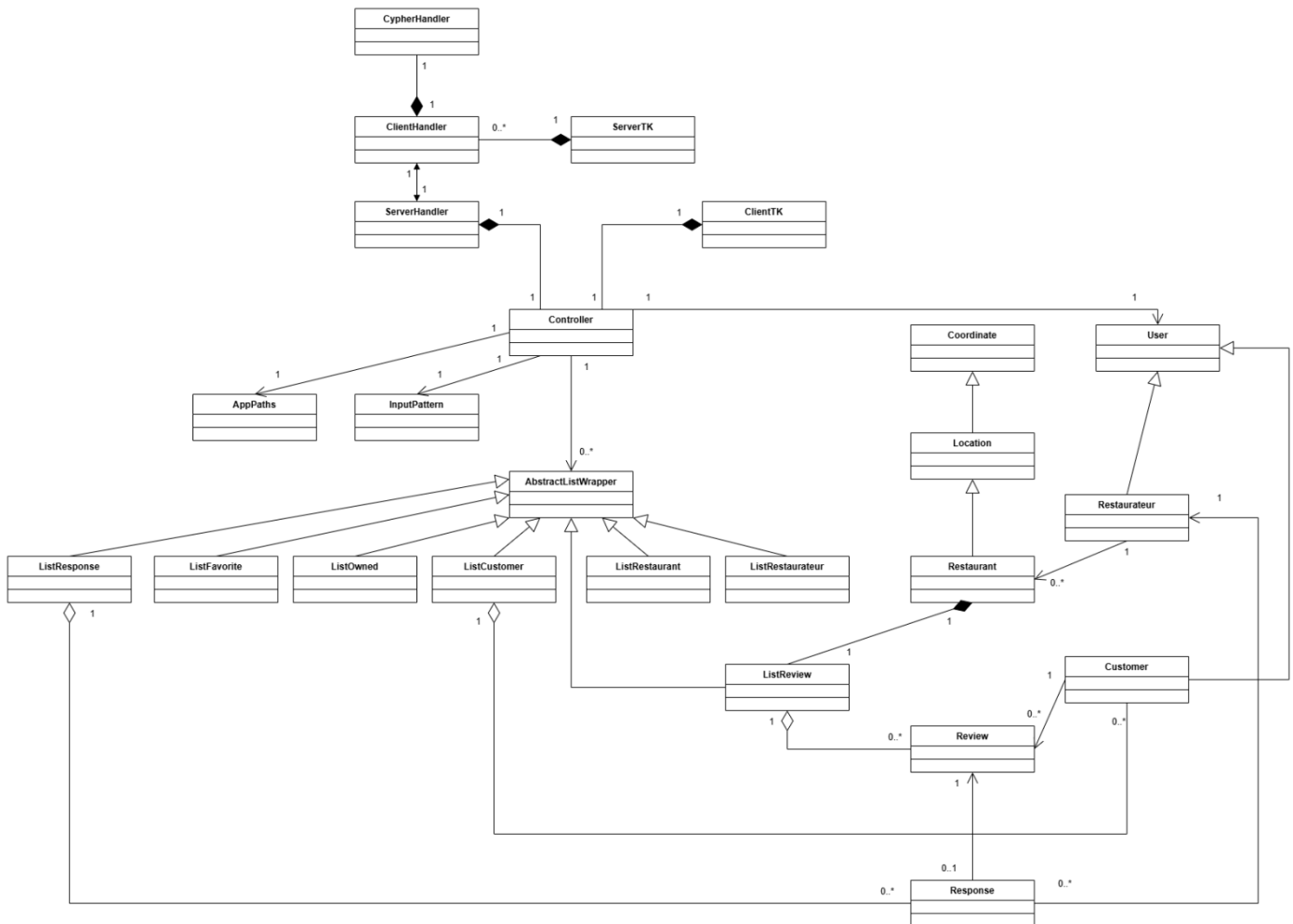
Response (ID (PRIMARY KEY), restaurateurID(FOREIGN KEY a Restaurateur(ID)), reviewID (FOREIGN KEY a Review(ID)), content)

E' inoltre necessario specificare alcuni vincoli in linguaggio letterale:

- Customer: controllo su birthDate che verifichi che l'età del cliente sia di almeno 13 anni;
- Restaurateur: controllo su birthDate che verifichi che il ristoratore sia maggiorenne;
- Restaurant: controllo su latitudine e longitudine che verifichi la correttezza delle informazioni (tra -90 e 90 e tra -180 e 180); controllo su rating che verifichi che il valore sia tra 0 e 5 estremi inclusi;
- Review: controllo su rating che verifichi che il valore sia tra 0 e 5 estremi inclusi.

2.5 Class diagram UML

Per garantire una migliore comprensione del sistema ad un livello di astrazione maggiore rispetto al codice stesso dell'applicazione, è stato costruito un class diagram di questo tipo:



In tale diagramma sono state omesse volontariamente le classi che non partecipano attivamente alla gestione della comunicazione tra client, server e database in quanto non sarebbe stato possibile garantire un buon livello di ordine generale. Le classi inerenti alla parte di interfaccia grafica proprio per questo motivo non compaiono nel diagramma.

Inoltre, attributi e metodi non sono stati segnalati in quanto il diagramma avrebbe raggiunto un livello di complessità e dimensione che avrebbe fatto perdere leggibilità allo schema andando così ad intaccare il requisito base di comprensibilità di un class

diagram UML. Nonostante ciò è comunque necessario specificare gli attributi di alcune classi fondamentali; in seguito un elenco:

User:

- username: String
- firstName: String
- role: String
- firstNameNormalized: String
- email: String
- password: String
- lastName: String
- birthDate: String
- address: String
- id: int
- lastNameNormalized: String

Customer:

- listFavorite: ListFavorite

Restaurateur:

- listOwned: ListOwned

Restaurant:

- description: String
- award: String
- price: int
- ownerId: int
- greenStar: boolean
- phoneNumber: String
- normalizedName: String
- listReview: ListReview
- currency: String
- name: String
- id: int
- url: String
- cuisine: String
- servicesAvailable: String
- websiteUrl: String
- rating: double

Location:

- city: String
- country: String
- address: String

Coordinate:

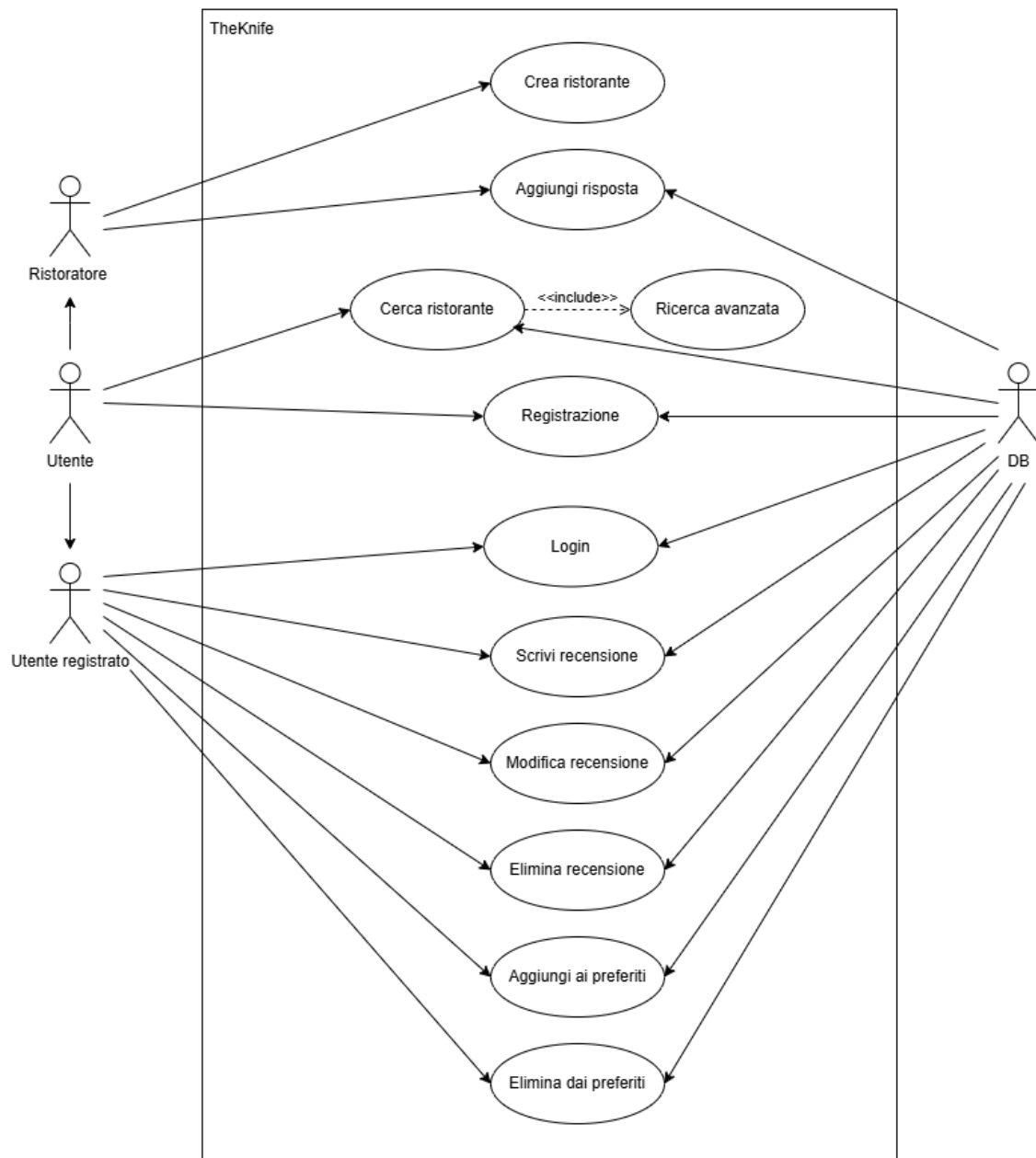
- latitude: double
- longitude: double

Review:

- content: String
- rating: double
- id: int
- restaurantID: int
- responses: ListResponse
- username: String

2.6 Use case diagram UML

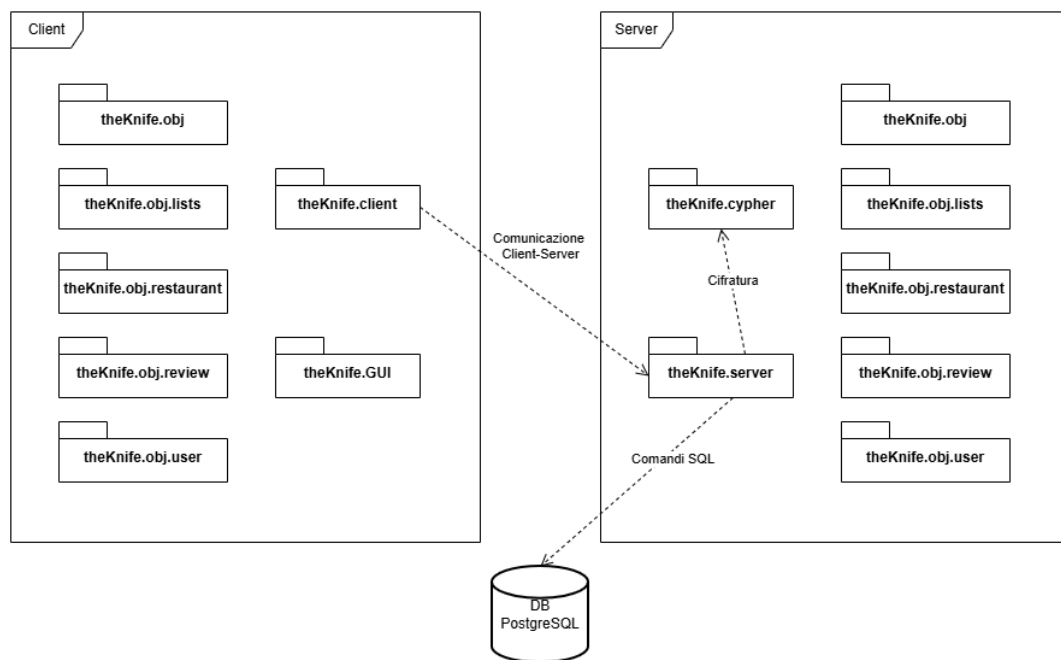
Per facilitare l'utilizzo e la comprensione delle funzionalità offerte dall'applicazione può risultare utile definire con precisione i casi di utilizzo e gli attori all'interno del sistema. Di conseguenza è stato sviluppato uno use case diagram così composto:



Non è presente tra gli attori il server in quanto viene utilizzato appositamente per permettere la comunicazione degli utenti (e le loro interazioni con il sistema) con il database. Inserirlo infatti avrebbe causato una ridondanza che sarebbe risultata fastidiosa in fase di osservazione del diagramma.

2.7 Package diagram UML

Il sistema a livello codice può risultare complesso in fase di lettura per il suo elevato numero di classi che comunicano tra di loro e di conseguenza per la presenza di molti diversi package che contengono e separano le diverse funzionalità del programma. Per questo motivo viene fornito in seguito un package diagram che mantiene comunque un certo margine di approssimazione per garantire una comprensione intuitiva e visiva delle interazioni/separazioni tra i diversi package che costituiscono il sistema.



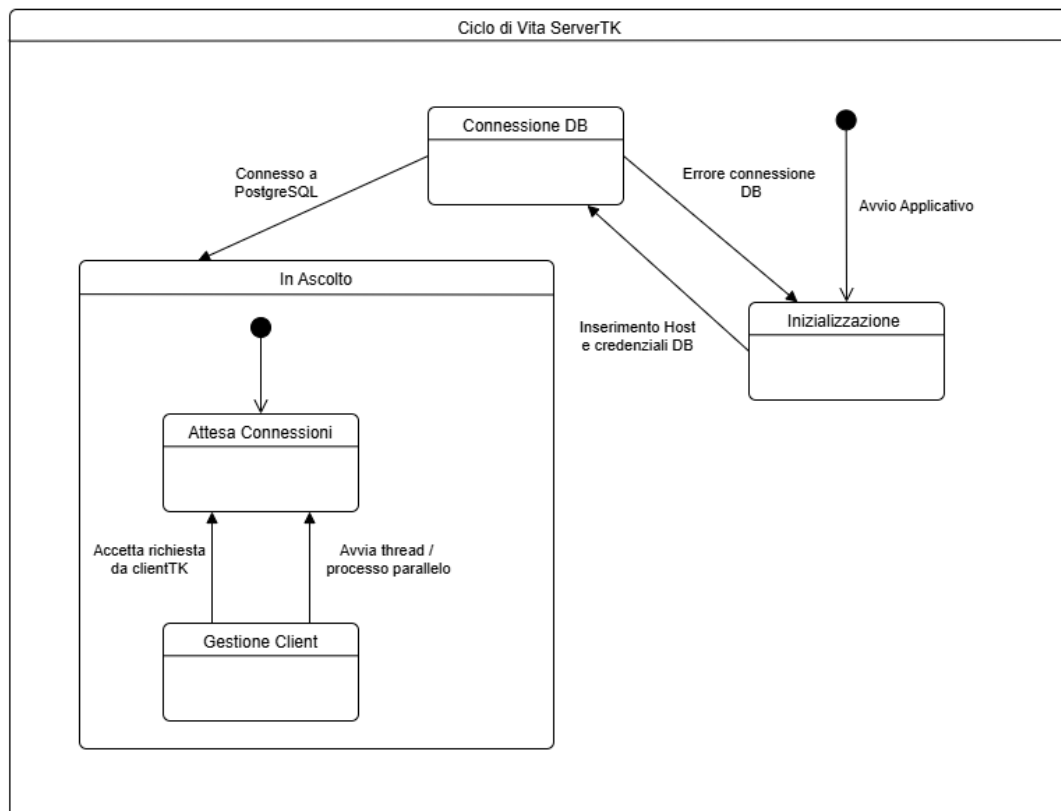
Nonostante non si tratti propriamente di un package vero e proprio, è stato raffigurato all'interno del diagramma anche il database PostgreSQL in quanto ha un ruolo importante nell'insieme del sistema. Il suo inserimento permette di avere un quadro più generale delle dipendenze tra i principali package dell'applicazione.

2.8 State diagram UML

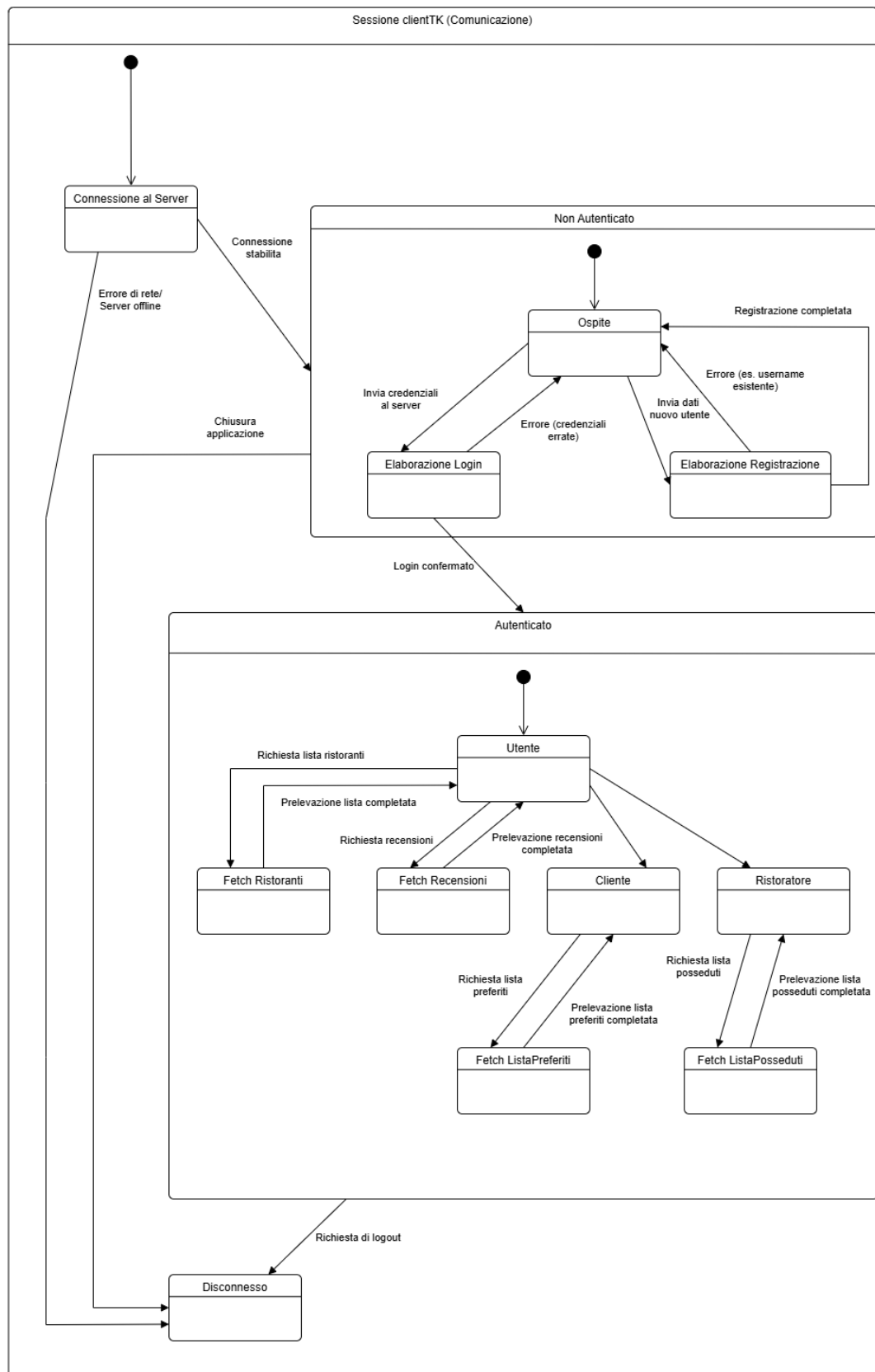
Per migliorare la comprensione del codice dal punto di vista degli stati previsti durante il funzionamento dell'applicazione è stato sviluppato il seguente state diagram.

Come per i precedenti diagrammi, sono state omesse parti del codice non estremamente rilevanti per rendere il tutto più leggibile mettendo in risalto i frangenti più importanti degli stati.

State diagram rappresentante il ciclo di vita del server:



State diagram rappresentante una classica sessione di attività del client:



Scelte architetturali

3.1 Interfaccia grafica

L'interfaccia grafica del programma è stata sviluppata utilizzando Swing, un framework per Java appartenente alla Java Foundation Classes che tramite l'implementazione di oggetti grafici quali pannelli, pulsanti e altro.

L'interfaccia si basa sull'utilizzo di diversi pannelli che vengono associati al pannello principale creato tramite la classe `PanelMain.java` che al suo interno sfrutta un oggetto `CardLayout`.

Tale oggetto permette di gestire tutti i pannelli creati (esempio: login, register, ecc...) come se fossero delle schede che vengono mostrate all'utente una alla volta in base alla richiesta senza che i pannelli debbano essere cancellati e ricreati diverse volte.

Per mostrare il pannello richiesto si sfrutta la funzione `getCard()` che posta nella funzione `showCard` all'interno della classe `PanelMain` permette di visualizzare la pagina richiesta in seguito ad un evento (ad esempio il click del mouse):

```
public final void showCard(String name) {  
    this.getCard().show(this.getPanel(), name);  
}
```

Mentre all'interno della funzione che gestisce il click del mouse (e con questo la richiesta dell'utente di accedere ad un'altra pagina/finestra del programma) si rimanda a tale funzione con il seguente comando:

```
pnl_main.showCard(Page.*nome pannello*);
```

Prendendo d'esempio il codice che crea gli oggetti inerenti alla creazione dei filtri per la ricerca avanzata mostrati nell'immagine, sono elencati di seguito gli altri oggetti grafici utilizzati all'interno del codice:

```
private void initFields() {  
    pnl_filters = new JPanel(new GridBagLayout());  
    pnl_btn_apply = new JPanel(new BorderLayout());  
    pnl_btn_cancel = new JPanel(new BorderLayout());  
    pnl_ratingBar = new JPanel(new GridLayout(1, RATINGS));  
    pnl_priceBar = new JPanel(new GridLayout(1, PRICES));  
    pnl_btms = new JPanel(new GridLayout(1, 2, 10, 10));  
    pnl_cuisines = new JPanel(new GridLayout((int) Math.ceil(CHEKBX_CUISINE_TXT.length / 2), 2, 10, 10));  
    pnl_services = new JPanel(new GridLayout((int) Math.ceil(CHEKBX_SERVICE_TXT.length / 2), 2, 10, 10));  
    scrPnl_filters = new JScrollPane(pnl_filters);  
    scrPnl_cuisines = new JScrollPane(pnl_cuisines);  
    lbl_title = new JLabel(TITLE);  
    lbl_stars = new JLabel[RATINGS];  
    lbl_prices = new JLabel[PRICES];  
    btn_apply = new JLabel(APPLY_FILTERS);  
    btn_cancel = new JLabel(CANCEL);  
    txt_location = new JTextField(PLACEHOLDER[0]);  
    chkbox_cuisines = new JCheckBox(CHEKBX_CUISINE_TXT.length);  
    chkbox_services = new JCheckBox(CHEKBX_SERVICE_TXT.length);  
    txt_layerUI = new RoundedComponentUI(ARC_TEXTFIELD);  
    btn_layerUI = new RoundedComponentUI(ARC_BUTTON);  
    pnl_layerUI = new RoundedComponentUI(ARC_PANEL);  
    scrPnl_filtersRounded = new JLayer<>(scrPnl_filters, pnl_layerUI);  
    txt_locationRounded = new JLayer<>(txt_location, txt_layerUI);  
    btn_applyRounded = new JLayer<>(btn_apply, btn_layerUI);  
    btn_cancelRounded = new JLayer<>(btn_cancel, btn_layerUI);  
}
```

ScrollPane:

pannello che a differenza della sua versione classica permette all'utente di scorrere il suo contenuto con il mouse; risulta utile quando si ha uno spazio limitato ma molto contenuto da rappresentare e non si vuole rendere troppo ingombrante il pannello (molto utile ad esempio per rappresentare i filtri di ricerca).

Label:

spazio che permette di aggiungere del testo statico in una posizione prestabilita dell'interfaccia.

TextField:

spazio che permette all'utente di inserire del testo; risulta utile nel caso in cui sia necessario l'inserimento di dati da parte dell'utente (ad esempio nel momento della registrazione dove si devono inserire i propri dati personali).

CheckBox:

bottone che può avere come valori solamente vero o falso, perciò si utilizza nel caso in cui sia necessaria una conferma a livello utente.

RoundedComponentUI:

permette di definire un arco di arrotondamento agli spigoli dell'oggetto scelto.

Layer:

permette di modificare la rappresentazione grafica di un oggetto dinamicamente durante l'utilizzo del programma; risulta infatti utile nel caso in cui sia necessario modificare il colore di un bottone o di una porzione di testo nel momento in cui vi si avvicina il cursore oppure in seguito ad un click del mouse.

PasswordField:

componente simile ad un normale TextField, con la differenza che permette di digitare un testo e nascondere i caratteri evitando così che possano comparire sullo schermo; per questo motivo è funzionale alla creazione di campi in cui devono essere inseriti dati sensibili da un utente come ad esempio una password.

ScrollBar:

componente che consiste in una semplice barra di scorrimento utile nel momento in cui sia necessario scorrere una pagina senza la necessità di avere un pannello direttamente associato (come nel caso dello ScrollPane).

Slider:

componente che permette di fare scorrere un cursore su una barra che generalmente presenterà dei valori da scegliere su cui fermarsi con il mouse. Utilizzato per esempio nel momento in cui, all'interno dei filtri di ricerca, si vuole scegliere la fascia di prezzo del ristorante.

GridBagLayout:

componente che suddivide la pagina sotto forma di griglia dove l'importanza delle diverse celle che contengono i componenti viene definita in maniera specifica di modo da fornire una corretta visualizzazione a qualsiasi risoluzione (questo perché alcune componenti devono rimanere più visibili rispetto ad altre).

Inoltre per facilitare la scrittura del programma sono stati introdotti dei componenti grafici personalizzati, implementati nelle classi: CustomJCheckBox, CustomJLabel, CustomJScrollBar e CustomJPanel. Le prime due citate si occupano in particolar modo di trattare dei caratteri speciali creati manualmente come immagini, mentre la classe CustomJScrollBar è stata creata per sovrascrivere il look and feel del tema nimbus. La classe CustomJPanel pone invece un gradiente allo sfondo della pagina.

3.2 Librerie

Per lo sviluppo del programma sono state usate numerose librerie offerte direttamente da Java (successivamente elencate), tuttavia per migliorare l'efficienza del progetto e la sua semplicità di sviluppo sono state utilizzate anche due librerie non comprese da Java: Simple e PgJDBC.

La libreria **Simple** è un package creato direttamente durante lo sviluppo del programma ed è fondamentale per aumentare l'efficienza riducendo la scrittura di codice e per aiutare il programmatore nella scrittura di certe funzioni che possono risultare lunghe o complesse senza una libreria costruita su misura. Tale libreria è portabile e generalizzata, perciò permette di essere utilizzata in altri progetti. Le principali funzioni che svolge Simple riguardano la cifratura con AES dei dati del database (in questo caso utilizzata per cifrare le credenziali dei clienti e dei ristoratori) e la normalizzazione di stringhe. Inoltre tale libreria svolge una funzione fondamentale nella comunicazione tra client e server rendendo più semplice lo scambio di oggetti e messaggi tra i due. Un esempio di utilizzo della libreria Simple può essere visto durante la registrazione di un cliente, dove il client invia al server il comando corrispondente al tentativo di registrazione che viene decifrato e dà il via al trasferimento degli oggetti (credenziali e informazioni personali).

La libreria **PgJDBC** è una API del linguaggio java che permette ad un client di interfacciarsi con il database di Postgre.

Le librerie già comprese da Java che sono state utilizzate sono delle seguenti tipologie (elenco suddiviso):

- **AWT** (Abstract Window Toolkit): gestione di interfacce grafiche
java.awt.BorderLayout;
java.awt.Color;

```
java.awt.Cursor;  
java.awt.Dimension;  
java.awt.Font;  
java.awt.GridBagConstraints;  
java.awt.GridBagLayout;  
java.awt.GridLayout;  
java.awt.Insets;  
java.awt.CardLayout;  
java.awt.Graphics;  
java.awt.Graphics2D;  
java.awt.RenderingHints;  
java.awt.Shape;  
java.awt.geom.RoundRectangle2D;  
java.awt.Rectangle;
```

- **IO** (Input/Output): gestione di operazioni di lettura e scrittura
java.io.File;
java.io.IOException;
- **UTIL**: implementazione delle strutture dati più comuni
java.util.HashMap;
java.util.List;
java.util.ArrayList;
java.util.Scanner;
- **SWING**: disposizione di un toolkit di sviluppo per la GUI portabile sui diversi SO
javax.swing.JPanel;
javax.swing.BorderFactory;
javax.swing.JComponent;
javax.swing.JLabel;
javax.swing.JLayer;
javax.swing.JTextField;
javax.swing.border.Border;
javax.swing.JCheckBox;
javax.swing.JScrollBar;
javax.swing.JScrollPane;
javax.swing.JSlider;
javax.swing.ScrollPaneConstants;
javax.swing.plaf.LayerUI;
javax.swing.JButton;
javax.swing.plaf.basic.BasicScrollBarUI;
- **SQL**: gestione delle query e della comunicazione con il database
java.sql.Connection;
java.sql.DriverManager;
java.sql.PreparedStatement;
java.sql.ResultSet;

`java.sql.SQLException;`

- **NET:** gestione della connessione e comunicazione tra lato client e server
`java.net.ServerSocket;`
`java.net.Socket;`

3.3 Strutture dati utilizzate

Nella cartella `theknife.obj.lists` troviamo le principali strutture dati utilizzate all'interno del programma.

AbstractListWrapper: classe che si occupa di creare una qualsiasi lista contenente un qualsiasi tipo di dato. La scelta di creare questa classe viene principalmente da una necessità di avere una struttura flessibile per facilitare il fetch e l'inserimento dei dati dal database.

A livello tecnico la classe si occupa della creazione di ogni tipologia di lista richiesta senza dover necessariamente ripetere delle righe di codice per generare una lista all'interno delle diverse classi. Ad esempio, la lista dei preferiti (`listFavorite`) e la lista dei ristoranti (`listRestaurant`) vengono create entrambe all'interno della classe `Wrapper` permettendo così una migliore efficienza del codice e del programma stesso. Inoltre per permettere alle proprie classi discendenti di poter essere trasmesse sui socket, è stata implementata l'interfaccia `Serializable` (che a sua volta permette la serializzazione degli oggetti).

ListCustomer: classe che si occupa della creazione di una lista contenente tutti i clienti (compresi i relativi dati personali)

ListOwned: classe che si occupa della creazione di una lista contenente tutti i ristoranti posseduti da un determinato ristorante.

ListRestaurateur: classe che si occupa della creazione di una lista contenente tutti i ristoratori registrati.

ListFavorite: classe che si occupa della creazione di una lista di ristoranti preferiti che ogni cliente ha la possibilità di scegliere. Non presenta metodi particolari in quanto viene utilizzata solo per inserire dei precisi ristoranti in una lista.

ListRestaurant: classe che si occupa della creazione di una lista contenente tutti i ristoranti.

ListReview: classe che si occupa della creazione della lista contenente le recensioni scritte dai clienti per i diversi ristoranti.

ListCuisines: classe che si occupa della creazione della lista contenente tutte le diverse tipologie di cucina.

ListServices: classe che si occupa della creazione della lista contenente tutti i diversi tipi di servizi che possono essere offerti.

ListResponse: classe che si occupa della creazione della lista contenente le risposte alle recensioni date dal ristorante.

Per lo sviluppo del programma l'unica struttura dati che si è scelto di utilizzare è la lista in quanto i dati da immagazzinare sono molto numerosi e se fossero stati salvati all'interno di un'altra struttura quale l'arrayList si sarebbe incorso in una problematica di gestione dello spazio in memoria. Perciò la scelta presa mira a ottimizzare l'efficienza del programma e a non appesantire inutilmente la RAM durante l'esecuzione.

3.4 Oggetti di supporto al database

Per evitare che il programma debba costantemente fare operazioni di I/O sul database sono stati creati degli oggetti che rappresentano i principali campi delle tabelle direttamente nel codice Java. Questo permette un accesso ai dati più rapido e agevolato che favorisce anche una maggiore velocità di risposta del programma alle richieste dell'utente. Nonostante i diversi vantaggi di questa scelta, tra i possibili contro abbiamo un aumento dello spazio di RAM occupato dal programma in esecuzione e l'impossibilità di un aggiornamento in tempo reale della lista dei ristoranti.

Gli oggetti di cui si parla si possono dividere in tre categorie principali (suddivise in diversi package): user, restaurant e review.

User: si occupa di dichiarare tutte le variabili corrispondenti ai campi delle tabelle del database fatta eccezione della lista dei preferiti del cliente e della lista dei ristoranti posseduti dal ristorante. Inoltre User viene esteso da Customer e Restaurateur.

Restaurant: dichiara tutte le variabili corrispondenti ai campi delle tabelle del database, più in particolare sono state create le classi Coordinate e Location che contengono tutte le informazioni inerenti alla posizione del ristorante.

Review: dichiara tutte le variabili corrispondenti ai campi delle tabelle del database creando una classe principale (Review) che contiene le informazioni necessarie riguardo le recensioni degli utenti.

3.5 Gestione degli errori e logging

Nella libreria Simple per gestire le eccezioni generate dall'utilizzo dei Socket e assicurarsi che non vi siano errori è stata implementata una classe di logging chiamata `LoggerUtils`. Tale classe nel momento in cui viene generata un'eccezione o un errore si occupa di stabilire il suo livello di gravità e di gestire l'invio di un messaggio che lo identifica.

Gli errori, o meglio i livelli di gravità degli errori, sono stati definiti nel seguente modo:

- **SEVERE**: errore critico; un fallimento (I/O) che compromette l'operazione.
- **WARNING**: avviso o anomalia; l'esecuzione continua, ma c'è qualcosa da controllare.
- **INFO**: informazione normale; indica i passi principali che il sistema sta compiendo.
- **CONFIG**: impostazioni; registra i parametri e l'ambiente di avvio.
- **FINE**: debug di base; traccia il flusso generale del codice.
- **FINER**: debug intermedio; utile per tracciare l'ingresso e l'uscita dai metodi.
- **FINEST**: debug estremo; registra ogni minimo dettaglio, come i valori nei cicli.

3.6 Cifratura dei dati

Per aumentare la sicurezza dei dati degli utenti è stato implementato un sistema di cifratura di tipo AES (Advanced Encryption Standard) con chiave a 256 bit sulle password.

Tale sistema di cifratura non è stato implementato direttamente all'interno del codice del programma, bensì sono state utilizzate delle funzioni presenti nella libreria Simple. Simple infatti permette di creare delle variabili di tipo "AES" che con i metodi `encrypt` e `decrypt` permettono di cifrare e decifrare la password. La cifratura (metodo `encrypt`) delle password avviene perciò sia nella fase di registrazione del cliente che del ristorante. La decifratura (metodo `decrypt`) invece è necessaria nel momento in cui un qualsiasi utente in fase di login inserisce la propria password. Password che per essere controllata deve ovviamente essere confrontata con quella precedentemente salvata nel database nella tupla corrispondente al tipo di utente che, in quanto già cifrata, deve essere prima decifrata per permetterne il confronto.

Oltre all'utilizzo dei metodi citati, per permettere la comunicazione con il DB utilizziamo delle particolari query sviluppate all'interno di una apposita classe denominata `CypherHandler` (contenente il richiamo alla chiave AES (`keystore.jks`) che si possono suddividere in due fasi:

- **Fase di cifratura classica:** viene utilizzata la query “SELECT id, password FROM customer/restaurateur” per estrarre tutta la lista degli utenti che in seguito viene passata tupla per tupla a due ArrayList che conterranno ID e password (nel frattempo ovviamente la password viene cifrata con il metodo definito precedentemente). In seconda fase vengono eseguite delle nuove query “UPDATE customer/restaurateur SET password = ? WHERE id = ?” (con i punti di domanda vengono poi sostituiti dalla password cifrata e l’ID dell’utente) appositamente create all’interno di un ciclo for e passate in blocco al database per evitare di saturare la connessione.
- **Fase di cifratura in registrazione:** il metodo di cifratura funziona nella stessa maniera appena spiegata, con la differenza che questo metodo permette di cifrare il singolo utente appena registrato. Viene perciò cifrata semplicemente la sua password che ritorna in mano al server.

Per quanto riguarda la chiave di cifratura AES, una volta generata viene successivamente salvata all’interno di un file di estensione JKS (Java KeyStore). In questo modo la chiave non risulta visualizzabile in chiaro anche se dovesse essere aperto il file.

La fase di decifratura (anch’essa sviluppata all’interno della classe CypherHandler) consiste principalmente nell’utilizzo della query “SELECT username, password FROM customer/restaurateur WHERE username = ?” che una volta eseguita se restituisce delle tuple procede decifrando la password confrontandola con quella data. Al termine di questa operazione, se le due password sono uguali il metodo ritorna true.

Inoltre è stato scelto AES come algoritmo di cifratura poiché mantiene una buona efficienza, non rallenta eccessivamente l’applicazione in quanto la fase di decifratura non richiede tempi particolarmente lunghi e offre una buona protezione dei dati grazie alla sua chiave da 256 bit.

Scelte algoritmiche

4.1 Login

La pagina di login permette all'utente di inserire una email o un username (scelto dall'utente) e una password per accedere al proprio profilo The Knife. Inoltre tramite due messaggi (creati con delle label) che fungono da link permette ad un utente non ancora registrato di farlo e ad un eventuale ristoratore di creare il suo profilo personale. Sono anche ovviamente presenti due semplici bottoni che permettono di effettuare il login una volta inseriti i dati richiesti oppure di uscire dalla pagina e ritornare alla home.

La classe presenta i seguenti metodi principali che permettono la corretta gestione degli eventi sulla pagina:

login_ComponentResized: metodo che si occupa del ridimensionamento di tutti i componenti della pagina.

txt_emailUsername_FocusGained: metodo che nel momento in cui si fa click con il mouse all'interno della TextField per l'inserimento dell'username permette di eliminare il messaggio scritto al suo interno e far comparire un cursore al suo interno.

txt_emailUsername_FocusLost: metodo che nel caso in cui "txt_emailUsername_FocusGained" sia attivo, toglie il focus sulla TextField username nel momento in cui viene cliccato un altro oggetto nella pagina. In questo modo, il messaggio che suggerisce l'inserimento di una mail o username ricompare.

txt_password_FocusGained: metodo che esegue le medesime funzioni di "txt_emailUsername_FocusGained" con l'unica differenza che viene applicato alla TextField predisposta per l'inserimento della password.

txt_password_FocusLost: metodo che esegue le medesime funzioni di "txt_emailUsername_FocusLost" con l'unica differenza che viene applicato alla TextField predisposta per l'inserimento della password.

txt_password_KeyTyped: metodo che nel momento in cui si vuole cancellare la password appena inserita utilizzando i tasti ctrl+a+canc/delete risolve una problematica generata a livello grafico.

txt_password_KeyPressed: metodo che verifica se l'utente ha cercato di selezionare l'intero contenuto della TextField adibita all'inserimento della password usando i tasti ctrl+a.

txt_password_KeyReleased: metodo che si occupa di salvare il contenuto della TextField password.

chkbx_seePassword_ItemStateChanged: metodo che si occupa di modificare lo stato della checkbox posizionata di fianco alla TextField password.

btn_login_MouseClicked: metodo che, una volta cliccato il bottone login, verifica la corrispondenza tra i dati inseriti dall'utente e quelli nel DB (sfruttando la classe controller). Inoltre nel caso in cui l'utente clicca il bottone senza aver inserito username, password oppure avendo inserito valori non validi (sia se data invalida la regex sia che le credenziali siano errate), le textField per username e password si colorano di rosso.

btn_login_MouseEntered: metodo che si occupa di segnalare graficamente il posizionamento del cursore sul bottone di login.

btn_login_MouseExited: metodo che nel momento in cui si sposta il cursore dal bottone di login ad una qualsiasi altra parte della pagina, si occupa di riportare graficamente il bottone alla sua rappresentazione grafica iniziale.

btn_cancel_MouseClicked: metodo che, una volta cliccato il bottone di cancellazione, riporta l'utente alla pagina home.

btn_cancel_MouseEntered: metodo che si occupa di segnalare graficamente il posizionamento del cursore sul bottone di cancellazione ("cancel").

btn_cancel_MouseExited: metodo che nel momento in cui si sposta il cursore dal bottone di cancellazione ad una qualsiasi altra parte della pagina, si occupa di riportare graficamente il bottone alla sua rappresentazione grafica iniziale.

btn_register_MouseClicked: metodo che si occupa di aprire la pagina di registrazione qualora venga cliccato il bottone contenente la frase: "Still not our customer yet? Sign up here!".

btn_register_MouseEntered: metodo che si occupa di segnalare graficamente il posizionamento del cursore sul bottone di registrazione.

btn_register_MouseExited: metodo che nel momento in cui si sposta il cursore dal bottone di registrazione ad una qualsiasi altra parte della pagina, si occupa di riportare graficamente il bottone alla sua rappresentazione grafica iniziale.

btn_loginAsRestaurateur_MouseClicked: metodo che si occupa di aprire la pagina di registrazione ristoratore qualora venga cliccato il bottone contenente la frase: "Are you restaurateur? Log in here!".

btn_loginAsRestaurateur_MouseEntered: metodo che si occupa di segnalare graficamente il posizionamento del cursore sul bottone di registrazione ristoratore.

btn_loginAsRestaurateur_MouseExited: metodo che nel momento in cui si sposta il cursore dal bottone di registrazione ristoratore ad una qualsiasi altra parte della pagina, si occupa di riportare graficamente il bottone alla sua rappresentazione grafica iniziale.

Dal momento che l'utente non per forza possiede già un account personale di The Knife, la pagina di login rimanda qualora sia necessario alle pagine che permettono la registrazione dell'utente e del ristoratore. Le tre classi che si occupano di tali "sotto-pagine" sono: **LoginRestaurateur**, **Register**, **RegisterRestaurateur** ed il loro

funzionamento, i metodi e la rappresentazione grafica risultano molto simili alla classe `login` stessa.

4.2 Home

La classe `home` si occupa prevalentemente della creazione della pagina home inizializzando i componenti principali che la formano.

`Home` contiene un singolo metodo principale (**`scrIPnl_restaurants_MouseWheelMoved`**) che si occupa di aumentare la velocità di scorrimento della `ScrollBar` del pannello che mostra il catalogo dei ristoranti.

Per ottimizzare lo sviluppo della pagina sono state create altre classi che lavorano a stretto contatto con `Home` quali: `PreviewRestaurant` e `Upperbar`. La classe **`PreviewRestaurant`** si occupa di mostrare i dati principali del ristorante che vengono visualizzati nel riquadro posto nella parte centrale della pagina (contenente nome, tasto “go to details” e altre informazioni).

La classe **`Upperbar`** risulta invece più articolata e importante per lo sviluppo della pagina home in quanto gestisce la parte alta della schermata dove sono presenti i principali componenti di ricerca e login.

Tale classe presenta i seguenti metodi principali:

`upperbar_ComponentResized`: metodo che si occupa di ridimensionare adeguatamente il pannello dell’`upperbar` e la barra di ricerca.

`txt_searchbar_KeyTyped`: metodo che pone un numero massimo di caratteri che possono essere scritti nella barra di ricerca (64 caratteri).

`txt_searchbar_KeyPressed`: metodo che verifica se l’utente ha cercato di selezionare l’intero contenuto della barra di ricerca usando i tasti `ctrl+a`.

`txt_searchbar_KeyReleased`: metodo che si occupa di gestire la cancellazione intelligente dei caratteri inseriti nella barra di ricerca qualora venga selezionato l’intero testo utilizzando i tasti `ctrl+a`. Permette inoltre nel caso in cui venga cliccato il tasto di `backspace` viene visualizzata una stringa vuota, invece nel caso in cui venga cliccato un tasto corrispondente ad un carattere viene visualizzato il carattere stesso.

Inoltre `Upperbar` contiene altri metodi event listener che si occupano della gestione di funzioni più banali come il click dei bottoni o il passaggio del cursore su di essi.

4.3 Restaurant GUI

La classe RestaurantGUI si occupa della gestione e della rappresentazione della pagina contenente tutte le informazioni riguardanti un ristorante.

La pagina mostra i dati inerenti al ristorante contenuti nel DB come: fascia di prezzo da 1 a 4 (con il prezzo che sale di pari passo con il numero che indica la fascia), location, indirizzo, numero di telefono, link del sito web, servizi offerti dal ristorante. Inoltre sono presenti una breve descrizione scritta che introduce il ristorante al pubblico e una sezione contenente le recensioni scritte dagli utenti con aggiunta di una fascia di valutazione espressa in stelle (da 0.5 a 5).

La classe presenta i seguenti metodi principali che permettono la corretta gestione degli eventi sulla pagina:

restaurantGUI_ComponentResized: metodo che si occupa di ridimensionare adeguatamente i pannelli che contengono i bottoni di aggiunta di una recensione e di ritorno alla home.

scrIPnl_reviews_MouseWheelMoved: metodo che si occupa di aumentare la velocità di scorrimento della ScrollBar del pannello contenente le recensioni.

btn_back_MouseClicked: metodo che nel momento in cui viene cliccato il bottone “back” riporta l’utente alla pagina home.

btn_back_MouseEntered: metodo che si occupa di segnalare graficamente il posizionamento del cursore sul bottone “back”.

btn_back_MouseExited: metodo che nel momento in cui si sposta il cursore dal bottone “back” ad una qualsiasi altra parte della pagina, si occupa di riportare graficamente il bottone alla sua rappresentazione grafica iniziale.

btn_addReview_MouseClicked: metodo che nel momento in cui viene cliccato il bottone di aggiunta di una recensione sposta l’utente alla pagina predisposta per l’aggiunta delle recensioni.

btn_addReview_MouseEntered: metodo che si occupa di segnalare graficamente il posizionamento del cursore sul bottone di aggiunta di recensioni.

btn_addReview_MouseExited: metodo che nel momento in cui si sposta il cursore dal bottone di aggiunta di recensioni ad una qualsiasi altra parte della pagina, si occupa di riportare graficamente il bottone alla sua rappresentazione grafica iniziale.

L’implementazione della pagina di informazioni riguardanti il ristorante è in gran parte gestita da questa classe, tuttavia per questioni di ottimizzazione è stata creata un’altra

classe che gestisce dei dettagli. **PreviewReview**, infatti, si occupa di mostrare all'interno di un riquadro le recensioni del ristorante.

4.4 InputPattern

Per semplificare il controllo delle stringhe inserite dall'utente nel momento in cui si registra (es. password, email, ...) o registra il proprio ristorante è stata implementata la classe InputPattern. Questa tramite l'utilizzo di specifiche regex verifica se i dati inseriti rientrano nei parametri richiesti dal programma.

FIRST_NAME:

```
public static final Pattern FIRST_NAME =  
Pattern.compile("^[\\p{L}][\\p{L}\\p{M}'\\- ]{1,98}[\\p{L}]$");
```

Parametri imposti dalla regex:

1. deve iniziare e finire con una lettera
2. al suo interno può contenere lettere, diacritici, apostrofi, trattini e spazi
3. lunghezza da 3 a 100 caratteri

LAST_NAME:

```
public static final Pattern LAST_NAME =  
Pattern.compile("^[\\p{L}']-{2,100}$");
```

Parametri imposti dalla regex:

1. può contenere lettere da qualsiasi alfabeto, apostrofi e trattini
2. non sono ammessi spazi o altri caratteri
3. lunghezza da 2 a 100 caratteri

ADDRESS:

```
public static final Pattern ADDRESS =  
Pattern.compile("(?=. {1,256}$)(\\d{1,5}[A-Za-  
z]?\\s?[\\p{L}\\s\\.'-]+|[\\p{L}\\s\\.'-]+\\s\\d{1,5}[A-Za-  
z]?)(,\\s?[\\p{L}\\s\\.'-]+){1,3}$");
```

La regex permette di validare solo indirizzi composti da:

1. numero civico (prima o dopo il nome della via)
2. nome della via
3. da 1 a 3 ulteriori componenti separati da virgole (città, province o paesi)

4. lunghezza massima 256 caratteri

EMAIL:

```
public static final Pattern EMAIL =  
Pattern.compile("(?=.{6,256}$)[\\w.-]+@[a-zA-Z\\d-]+\\.([a-zA-Z]{2,}$)");
```

La regex valida solo email con:

1. corpo composto da lettere, numeri, underscore, punti e trattini
2. dominio (che inizia con @) composto da lettere, numeri, trattini
3. TLD (almeno 2 lettere)
4. Lunghezza da 6 a 256 caratteri

USERNAME:

```
public static final Pattern USERNAME =  
Pattern.compile("(?![_.])(?!.*\\.\\.)(?!.*____)[a-zA-Z0-9._]{3,32}(?![_.]$)");
```

La regex valida solo username che seguono tali parametri:

1. composto solo da lettere, numeri, punti e underscore
2. non può iniziare né finire con punti o underscore
3. non può contenere doppio punto
4. non può contenere 4 underscore consecutivi
5. lunghezza da 3 a 32 caratteri

PASSWORD:

```
public static final Pattern PASSWORD =  
Pattern.compile("(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@#$!%*?&\\(\\)])[a-zA-Z\\d@#$!%*?&\\(\\)\\.]{8,128}$");
```

La regex valida solo password che seguono tali parametri:

1. deve contenere almeno una maiuscola, una minuscola, una cifra e un carattere speciale tra @#\$!%*?&()
2. può contenere solo lettere, numeri e i caratteri speciali sopra citati
3. lunghezza da 8 a 128 caratteri

RESTAURANT_NAME:

```
public static final Pattern RESTAURANT_NAME
= Pattern.compile("^([\\p{L}\\p{N}][\\p{L}\\p{N}']&.,\\-
]{1,49}$");
```

La regex valida solo nomi che seguono tali parametri:

1. lunghezza massima di 50 caratteri
2. può contenere lettere, numeri e segni di punteggiatura
3. deve cominciare con una lettera o un numero

RESTAURANT_ADDRESS:

```
public static final Pattern RESTAURANT_ADDRESS
= Pattern.compile("(?=[1,256]$)(\\d{1,5}[A-Za-
z]?\\s?[\\p{L}\\s\\.'-]+|[\\p{L}\\s\\.'-]+\\s\\d{1,5}[A-Za-
z]?)(,\\s?[\\p{L}\\s\\.'-]+){1,3}$");
```

La regex valida solo indirizzi che seguono tali parametri:

1. composto da civico e nome via (in ordine libero) + da 1 a 3 località (separate da virgole)
2. lunghezza massima di 256 caratteri

RESTAURANT_COUNTRY & RESTAURANT_CITY:

```
public static final Pattern RESTAURANT_COUNTRY
= Pattern.compile("(?=[2,70]$)\\p{L}(?:[\\p{L}'\\-
]*\\p{L})$");

public static final Pattern RESTAURANT_CITY
= Pattern.compile("(?=[2,70]$)\\p{L}(?:[\\p{L}'\\-\\.
]*\\p{L})$");
```

La regex valida solo nomi che rispettano tali parametri:

1. lunghezza da 2 a 70 caratteri
2. può contenere lettere, apostrofi, trattini e spazi

3. deve iniziare e finire con una lettera

RESTAURANT_LATITUDE_AND_LONGITUDE:

```
public static final Pattern RESTAURANT_LATITUDE_AND_LONGITUDE
= Pattern.compile("^\\d\\.\\d+\\-]{1,15}$");
```

La regex verifica che:

1. sia stato inserito un numero decimale (con parte decimale preceduta da un punto)
2. accetta “+” e “-” prima del numero

RESTAURANT_CURRENCY:

```
public static final Pattern RESTAURANT_CURRENCY
= Pattern.compile("^\\p{Sc}]$");
```

La regex verifica che la stringa inserita:

1. sia lunga 1 carattere
2. contenga un simbolo di valuta

RESTAURANT_PHONE_NUMBER:

```
public static final Pattern RESTAURANT_PHONE_NUMBER
= Pattern.compile("^\\+?[\\d\\s\\-\\(\\)\\.]{7,20}$");
```

La regex valida solo numeri di telefono che come caratteri hanno cifre, spazi, trattini, parentesi tonde, punti e il + come carattere iniziale.

RESTAURANT_URL:

```
public static final Pattern RESTAURANT_URL
= Pattern.compile("^(https?):\\/\\/(www\\.)?([a-zA-Z0-9_-]+\\.)+[a-zA-Z]{2,}(:\\d{1,5})?(\\/[^\s?#]*)?(\\?[^\s#]*)?(#\\S*)?$");
```

La regex valida solo URL che rispettano i seguenti parametri:

1. la stringa deve iniziare con https:// o http://, può avere www.
2. segue il dominio (o più segmenti del dominio divisi dal punto) composto da lettere, numeri, underscore o trattino
3. il dominio deve terminare con il TLD (.it, .com, ecc...), dunque almeno due lettere
4. può avere una porta composta da due punti e 4 cifre
5. può avere un path, dunque preceduto da slash accetta un qualsiasi carattere tranne spazio e #

6. può avere una query string, dunque preceduto da ? accetta un qualsiasi carattere tranne spazio e #
7. può avere un fragment/hash, dunque accetta qualsiasi carattere tranne spazio preceduto da #

RESTAURANT_WEBURL:

```
public static final Pattern RESTAURANT_WEBURL
= Pattern.compile("^(https?):\\/(\\/((([a-zA-Z0-9_-]+\\.)+[a-zA-Z]{2,}))\\/[^\s]*)?$");
```

La regex valida solo WEBURL che rispettano i seguenti parametri:

1. inizio con http:// o https://
2. seguito da un dominio valido (con TLD di almeno 2 lettere)
3. può avere un percorso (path) dopo il dominio
4. non consente query string (?) o frammenti (#)

4.5 Controller

La classe controller si occupa della gestione dati e di interfacciare la GUI con i dati contenuti nel database. Più in particolare tale classe si appoggia alla classe `ServerHandler` per recuperare i dati dal database e inserirli nelle rispettive liste create appositamente per avere tutte le risorse necessarie in loco.

I principali metodi che offre la classe `Controller` sono:

LoginClient: metodo che verifica se username e password inseriti dall'utente in fase di login rispettano i parametri stabiliti dalle regex e successivamente verifica che i dati inseriti siano presenti nel database. Infine se l'utente esiste viene inserito in `loggedUser`.

LoginRestaurateur: metodo che si comporta come il metodo "LoginClient" con l'unica differenza che gestisce i profili dei ristoratori.

RegisterClient: metodo che si occupa di verificare se l'username inserito è già stato utilizzato da un altro utente e qualora questo venga accettato procede con l'aggiunta alla lista dei clienti nel database.

RegisterRestaurateur: metodo che si comporta come il metodo "RegisterClient" con l'unica differenza che gestisce la registrazione dei ristoratori.

addRestaurant: metodo che si occupa di aggiornare la lista nel database predisposta all'aggiunta di un ristorante da parte di un ristorante.

searchRestaurant: metodo che riceve il contenuto scritto nella upperbar dall'utente e lo invia alla classe home che si occuperà di trovare le corrispondenze inerenti alla stringa data.

viewUserList: metodo che verifica se l'utente che richiede la lista è un cliente o un ristorante (comunicandolo alla classe home) e successivamente invia alla classe home la lista dei ristoranti preferiti se si tratta di un cliente, mentre invia la lista dei ristoranti posseduti se si tratta di un ristorante.

advancedSearch: metodo che si occupa di filtrare la lista dei ristoranti seguendo le specifiche selezionate dall'utente. Più in particolare, viene creata una lista "copia" della lista dei ristoranti che sfruttando i metodi della classe List viene modificata eliminando tutti i ristoranti che non rispettano le caratteristiche. Preliminarmente però quando il programma viene aperto, dopo aver recuperato i dati dal database, si occupa di generare delle HashMap per ogni filtro; HashMap che sono utili nel nostro caso per suddividere i ristoranti in base ai parametri e rendere più facile la loro eliminazione dalla lista.

initLists: metodo che si occupa di prelevare dal database la lista dei ristoranti, dei tipi di cucina e dei servizi tramite la classe ServerHandler. Risulta utile durante l'esecuzione dell'applicazione per avere tutti i dati inerenti ai ristoranti in loco.

4.6 Aggiunta di recensioni

L'aggiunta di recensioni è una funzionalità che il programma offre a tutti i clienti che hanno effettuato il login.

I due principali metodi che permettono la creazione di una recensione all'interno della classe **AddReview** sono:

lbl_star_MouseClicked: metodo che si occupa di salvare il numero di stelle che l'utente ha selezionato e, una volta cliccato sulla valutazione scelta, mantiene colorate le stelle fino a che non venga aggiunta o cancellata la recensione.

btn_add_MouseClicked: metodo che nel momento in cui viene cliccato il bottone di "add" si occupa di salvare la recensione all'interno dell'apposita lista (con annessi i dati del ristorante e del cliente che l'ha scritto).

Invece il principale metodo all'interno della classe **AddResponse** che permettono la creazione di una risposta da parte del ristorante è:

btn_add_MouseClicked: metodo che nel momento in cui viene cliccato il bottone di “add” si occupa di salvare la risposta all’interno dell’apposita lista (con annessi dati del ristorante e del ristoratore).

4.7 Server

Il lato server dell’applicazione viene gestito all’interno del package `theKnife.server` che a sua volta contiene le tre classi:

ServerTK: classe che si occupa dell’avvio del server e dell’apertura di una comunicazione con il database (tutto ciò avviene grazie al metodo **`startServerServices`**). In seguito a queste prime due operazioni rimane in ascolto per eventuali richieste di connessione da parte di un client.

ClientHandler: classe thread che apre la connessione tra server e client e si occupa di decodificare i comandi inviati dal client. Per prima cosa la classe presenta un valore enumerativo contenente i nomi di tutti i comandi che possono essere inviati dal client verso il server. La lista di tali comandi è la seguente:

- **LOGIN_CUSTOMER:** permette di far accedere un cliente;
- **LOGIN_RESTAURATEUR:** permette di far accedere un ristoratore;
- **REGISTER_CUSTOMER:** permette di far registrare un cliente;
- **REGISTER_RESTAURATEUR:** permette di far registrare un ristoratore;
- **GET_RESTAURANTS:** permette di prelevare dal database la lista dei ristoranti presenti;
- **GET_CUISINES:** permette di prelevare dal database la lista dei tipi di cucina esistenti;
- **GET_SERVICES:** permette di prelevare dal database la lista dei servizi esistenti;
- **GET_REVIEWS:** permette di prelevare dal database la lista delle recensioni del ristorante visualizzato;
- **ADD_FAVOURITE:** permette di inserire (come cliente) un nuovo ristorante nella lista dei preferiti;
- **REMOVE_FAVOURITE:** permette di cancellare (come cliente) un ristorante dalla lista dei preferiti;
- **ADD_RESTAURANT:** permette di inserire (come ristoratore) un nuovo ristorante al database;
- **ADD_REVIEW:** permette di creare (come cliente) una recensione ad un ristorante;
- **EDIT_REVIEW:** permette di modificare (come cliente) una recensione scritta dal cliente che richiede di modificare;
- **ADD_RESPONSE:** permette di creare (come ristoratore) una risposta ad una recensione

- **REMOVE_REVIEW**: permette di cancellare (come cliente) una recensione scritta dal cliente che richiede la cancellazione;

Per quanto riguarda invece la funzione `run` della classe, in questa fase si attende la ricezione di un comando che successivamente viene decodificato (ovviamente in base al comando ricevuto, il server potrebbe rimanere ulteriormente in attesa se necessario). I comandi decodificati fanno scattare la richiesta al database tramite metodi che definiscono l'intera procedura di invio della query e gestione del risultato ottenuto. Un esempio possono essere i metodi ***getRestaurant*** e ***getFavorite*** che restituiscono gli oggetti contenenti i dati richiesti.

Inoltre la classe contiene tutte le query disponibili per la comunicazione tra client e database; viene invocata nel momento in cui un client fa una particolare richiesta (ad esempio visualizzare i ristoranti) e fornisce la corretta query per trovare i dati specifici cercati dall'utente oppure per fornirli al database in caso di input.

Risulta particolarmente rilevante all'interno della classe (e può essere necessario approfondire in quanto la query al suo interno potrebbe non essere banale) il metodo ***getRestaurants*** che contiene la query necessaria per restituire i dati inerenti a tutti i ristoranti al client e la conseguente chiamata al database:

```
"SELECT" +
"    r.*, " +
"    CASE" +
"        WHEN COUNT(c.id) = 0 THEN '' +
"        ELSE STRING_AGG(DISTINCT c.description, ', ')" +
"    END AS cuisines," +
"CASE" +
"        WHEN COUNT(s.id) = 0 THEN '' +
"        ELSE STRING_AGG(DISTINCT s.description, ', ')" +
"    END AS service" +
"FROM restaurant r" +
"LEFT JOIN listcuisine lc" +
"    ON lc.restaurantId = r.id" +
"LEFT JOIN cuisine c" +
"    ON c.id = lc.cuisineId" +
"LEFT JOIN listservice ls" +
"    ON ls.restaurantId = r.id" +
"LEFT JOIN service s" +
"    ON ls.serviceId = s.id" +
"GROUP BY r.id;;
```

Come si può notare leggendo la query mostrata viene selezionata tutta la colonna dei ristoranti gestendo le tipologie di cucina e i servizi. Per inserire tutti i dati utili in una singola tupla è stata usata la funzione `CASE` insieme alla funzione `STRING_AGG` che insieme permettono di estrarre le due liste `cuisines` e `service` ottenendo in questo modo le tipologie di cucina e i servizi offerti dal singolo ristorante e posizionati in due colonne distinte.

Per prevenire che utenti malintenzionati sfruttassero la SQL Injection per manipolare il database è stata implementata la libreria `java.sql.PreparedStatement`. Essa permette di eseguire query SQL precompilate e parametrizzate, (ovvero i parametri vengono inviati separatamente dalla query e trattati rigorosamente come dati), neutralizzando qualsiasi tentativo di iniezione. Questa libreria permette oltretutto di ottimizzare i tempi di esecuzione, in quanto, se deve essere eseguita la stessa query più volte ma con parametri diversi, il db non dovrà rianalizzare la query, ma solo cambiarne i parametri.

4.8 ServerHandler

La classe `thread serverHandler`, facente parte del package `client`, si occupa di comunicare al server per prima cosa la necessità del client di dare il via ad una comunicazione ed in seguito i comandi e o le operazioni richieste.

Il principale metodo usato da `serverHandler` è:

sendData: metodo che tramite l'utilizzo del metodo `send (Socket, T)` dove "T" è un tipo generico offerto dalla libreria `Simple` si occupa di inviare un oggetto al server. Tale metodo può risultare molto utile in quanto permette di inviare gli oggetti più utilizzati all'interno dell'applicazione (come ristoranti, clienti, recensioni e ristoranti) con facilità.

`ServerHandler` inoltre contiene tutti i metodi necessari per la comunicazione del client con il server. Esiste infatti un metodo per ogni comando che può essere richiesto di eseguire e che necessita dell'intervento del server.

Formato dei file e gestione

5.1 Formato file

Tramite l'utilizzo dei tool offerti da PostgreSQL è stato possibile popolare le tabelle del database semplicemente passandogli il file CSV contenente il dataset precedentemente ordinato con l'utilizzo del linguaggio Python.

I file di codice invece presentano tutti il formato java in quanto è l'unico linguaggio di programmazione utilizzato per lo sviluppo del programma.

5.2 AppPaths

Per semplificare il recupero delle risorse necessarie al funzionamento del programma dalla cartella contenente il file eseguibile (jar) è stata creata la classe AppPaths.

L'obiettivo di tale classe è ottimizzare l'accesso alla cartella jar per arrivare in maniera più semplice a ottenere le immagini utilizzate nello sviluppo grafico del programma, il font (che si trova anch'esso nella cartella img) e la chiave di cifratura AES.

5.3 Dataset di test

Per effettuare dei test di funzionamento del programma è stato utilizzato un dataset principale contenente un alto numero di ristoranti, tuttavia è stato necessario estendere tale dataset creandone altri contenenti alcuni profili creati ad hoc sia di ristoratori che di clienti. Inoltre per verificare la correttezza di alcune funzioni implementate si è dimostrato necessario implementare all'interno del dataset dei ristoranti una lista di recensioni generate automaticamente.

La generazione dei dataset riguardanti ristoratori e clienti è stata svolta utilizzando il linguaggio Python, più in particolare sfruttando la libreria Faker (che lavora con un database contenente diversi nomi e cognomi per ogni nazionalità scelta e anche possibili date di nascita).

La generazione delle recensioni è stata anch'essa svolta in linguaggio Python. In questo caso però per rendere più dettagliate e reali le recensioni è stato deciso di creare tre diversi array per separare le recensioni positive, neutrali e negative (aumentando così la coerenza del voto dato). Inoltre per correttezza dei dati è stato preso ID dalla tabella Customer contenente i clienti per ogni recensione scritta.

Qualora si volessero verificare personalmente i dati aggiunti, questi possono essere trovati all'interno della cartella "data" del programma sotto forma di file CSV.

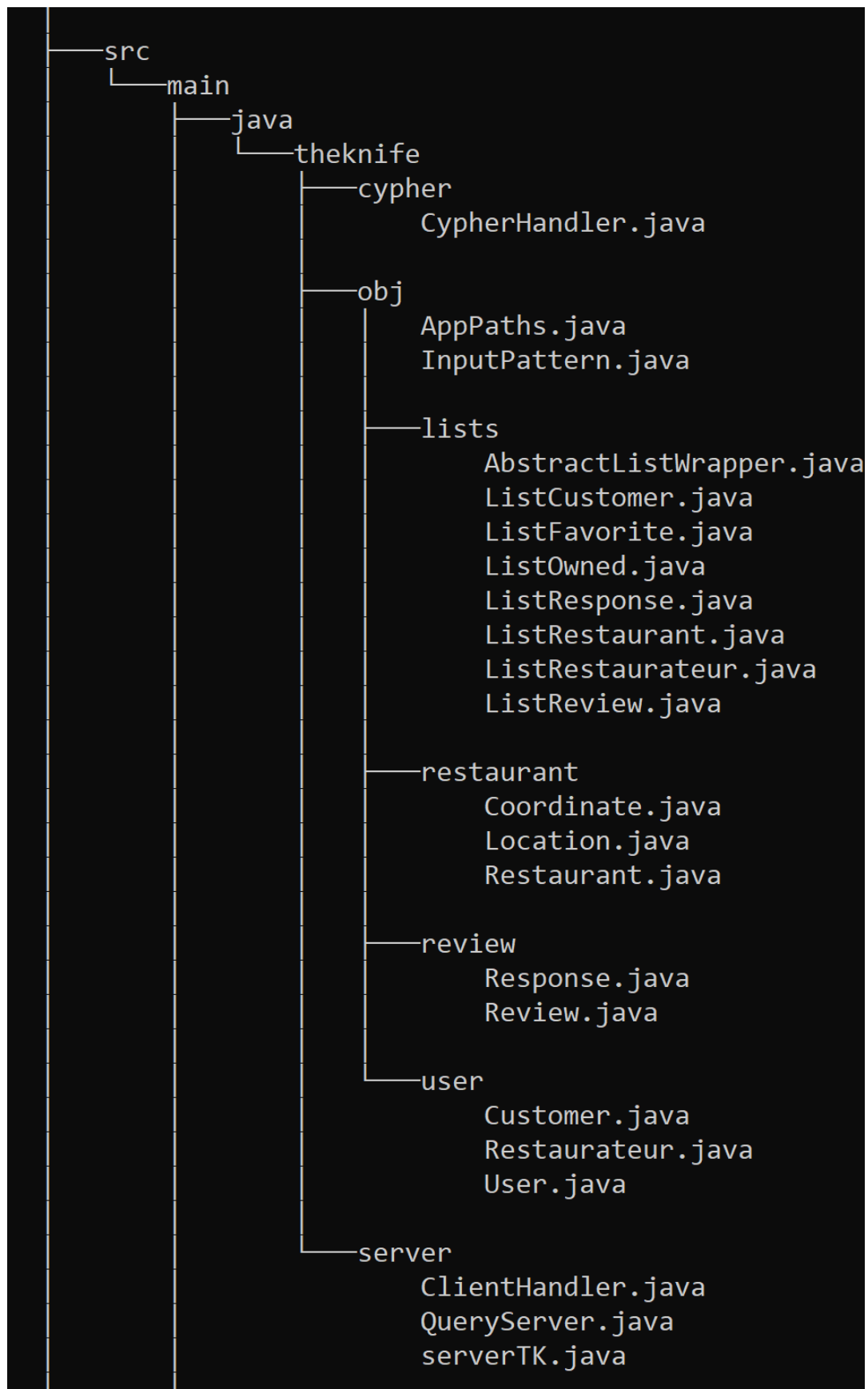
5.4 Gestione dei file



Nell'immagine è rappresentato il sistema di gestione dei file del programma sfruttando una struttura ad albero.

Sono ben visibili le principali suddivisioni di file come theknife, gui e obj. Queste contengono rispettivamente la logica principale del programma, le strutture che formano l'interfaccia grafica e gli oggetti a cui vengono tipizzate le liste (visibili nella sotto-suddivisione lists).

Nella pagine seguente è inoltre visibile anche la rappresentazione del lato server sempre con la stessa notazione.



Limitazioni della soluzione sviluppata

Il sistema non presenta grosse limitazioni dal punto di vista di utilizzo in quanto si tratta di una soluzione client-server collegato ad un proprio database. Tuttavia per quanto riguarda la cifratura delle comunicazioni tra client e server potrebbe essere ulteriormente migliorata la sicurezza utilizzando una cifratura di tipo AES + RSA. Questa scelta però è stata evitata per non complicare esageratamente il codice e per mantenere una discreta velocità di cifratura e decifratura. L'utilizzo congiunto di AES e RSA infatti avrebbe allungato in maniera non indifferente i tempi delle fasi descritte in precedenza aumentando il rischio di ottenere un'applicazione lenta e di utilizzo scomodo per l'utente.

Un'altra piccola limitazione dal punto di vista di utilizzo (come accennato nei capitoli precedenti) è l'impossibilità di un aggiornamento in tempo reale della lista dei ristoranti. Seppur si tratti di una soluzione client-server collegato ad un proprio database, è stata presa la decisione di fare una singola prelevazione della lista dei ristoranti nel momento dell'apertura del programma. Tutto ciò è stato fatto con lo scopo di ridurre la congestione del traffico tra client e server tenendo conto anche del fatto che l'applicazione presenta già un buon algoritmo di ricerca dei ristoranti il locale, dunque non è sembrato opportuno ristrutturare la ricerca rischiando di affaticare ulteriormente la connessione.